專題研究(一)進度報告-進度3

專題名稱：視覺化網路攻擊自動化分析平台
成員：D1378300 胡安硯、D1300043 何昇駿、D1256832 石任立

一、工作與進度報告

(本次進度：消融實驗與模型壓縮評估)

目前程式的流程圖

一、資料擷取與前處理模組
LiveCapture 即時監聽 / PCAP 載入
PacketParser 封包解析
提取網路特徵與流量分析
轉換為 32x32 矩陣/影像特徵
即時/批次測試資料
正常與攻擊流量集
二、訓練與模型剪枝模組
CNN Autoencoder 模型
訓練模式選擇
半監督式
非監督式
引入 MarginLoss 推高攻擊誤差
使用 MSE Loss 學習重建正常流量
計算 Reconstruction Error
閾值調校 Threshold Tuning
消融實驗 Ablation Study
模型壓縮: 知識蒸餾 / 剪枝 / 量化
參數大幅縮減至約 300K
匯出最佳模型 best_model.pt
三、線上偵測與 Django 平台
Django 載入 CNN 權重與設定檔
計算封包即時重建誤差
誤差 > CNN_THRESHOLD ?
否
是
判定為正常 Normal
判定為異常 Attack
統計 Session 分析數據
建立 Alert 告警紀錄
Django 資料庫
後端 Admin 與前端儀表板視覺化呈現

消融實驗模組 (ablation_study.py)

針對 CNN Autoencoder 設計七個控制變因的消融實驗，分別為潛在空間維度、架構縮放比例、輸入影像尺寸、封包欄位遮罩策略、訓練資料量、激活函數類型，以及訓練策略比較 (半監督微調與純非監督的比較)。為支援多樣架構測試，另行實作 CNNAutoencoderFlex 彈性模型，支援可變通道數、激活函數（ReLU / GELU / LeakyReLU）與任意輸入尺寸（透過 AdaptiveMaxPool2d 固定瓶頸為 4 x 4）。實驗結束後自動產生摘要圖表並輸出 JSON 報告

核心程式邏輯如下：

```
# core/ablation_study.py

class CNNAutoencoderFlex(nn.Module):
    """可調整架構的 CNN Autoencoder，用於消融實驗"""
    def __init__(self, latent_dim=32, channels=None,
                 activation="relu", image_size=32):
        ch = channels or [16, 32, 64, 64]
        # Encoder：AdaptiveMaxPool2d 固定瓶頸為 4×4，支援任意輸
入尺寸
        self.encoder_conv = nn.Sequential(
            ...,
            nn.AdaptiveMaxPool2d((4, 4)),
        )
        # Decoder：Upsample 精確對齊目標尺寸，解決非 2^n 尺寸問題
        self.decoder_deconv = nn.Sequential(
            ...,
            nn.Upsample(size=(image_size, image_size), mode
="bilinear"),
            nn.Conv2d(ch[0], 1, 3, padding=1), nn.Sigmoid
(),
        )

class AblationStudy:
    def _run_single(self, name, model, X_train, X_normal_ev
al, X_attack_eval):
        """訓練模型並評估 F1 / AUC / Precision / Recall"""
        _quick_train(model, X_train, self.epochs, ...)
```

```
        errors_n = _compute_errors_np(model, X_normal_eval,
self.device)
        errors_a = _compute_errors_np(model, X_attack_eval,
self.device)
        return {"name": name, "params": _count_params(mode
l),
                **_evaluate_threshold(errors_n, errors_a, s
elf.percentile)}

    def run_latent_dim(self, latent_dims=[4, 8, 16, 32, 6
4]):
        for ld in latent_dims:
            model = CNNAutoencoderFlex(latent_dim=ld)
            res = self._run_single(f"latent={ld}", model,
...)

    # 其餘五個實驗的做法相似，考量報告的篇幅，只放部分內容
```

訓練策略比較模組 (comparison_benchmark.py)

實作 ComparisonBenchmark 類別，將純非監督式訓練與半監督式微調訓練進行並排比較，評估指標包含 Precision、Recall、F1、AUC-ROC 及分離比（Separation Ratio），並繪製雙欄對比圖表與輸出 benchmark_report.json報告

核心程式邏輯如下：

```
# core/comparison_benchmark.py

class ComparisonBenchmark:
    def run(self):
        results = {}
        results["unsupervised"]    = self._run_unsupervised
()   # Phase 1 only
        results["semi_supervised"] = self._run_semi_supervi
sed() # Phase 1 + 2
        return results
```

```
    def _run_unsupervised(self):
        # 僅以正常流量訓練，不使用任何攻擊樣本
        trainer.load_data(normal_npy)
        trainer.train()
        err_n = _compute_errors_np(trainer.model, self.X_no
rmal, ...)
        err_a = _compute_errors_np(trainer.model, self.X_at
tack, ...)
        return {"sep_ratio": err_a.mean() / (err_n.mean() +
1e-9),
                **_evaluate_threshold(err_n, err_a)}

    def _run_semi_supervised(self):
        # Phase 1 預訓練 + Phase 2 以少量攻擊樣本做邊界損失微調
        trainer.train_full(self.X_normal, self.X_attack)
        err_n = _compute_errors_np(trainer.model, self.X_no
rmal, ...)
        err_a = _compute_errors_np(trainer.model, self.X_at
tack, ...)
        return {"sep_ratio": err_a.mean() / (err_n.mean() +
1e-9),
                **_evaluate_threshold(err_n, err_a)}

    def plot_all(self, results):
        # 左圖：Precision / Recall / F1 / AUC 雙色長條圖對比
        # 右圖：分離比（Separation Ratio）對比
```

模型壓縮模組 (model_compression.py)

實作 ModelCompressor 類別，整合 UnstructuredPruner、StructuredPruner、PostTrainingQuantizer、KnowledgeDistiller 四個子類別，支援非結構化剪枝、結構化剪枝、訓練後靜態量化（PTQ INT8）以及知識蒸餾四種壓縮方法。每種方法均記錄壓縮率、參數量、模型大小、F1、AUC-ROC 及推論延遲，並與未壓縮 Baseline 進行比較，最終輸出 Trade-off 曲線圖與 compression_report.json報告。

核心程式邏輯如下：

```
# core/model_compression.py

class KnowledgeDistiller:
    def __init__(self, teacher_model, student_latent_dim=1
6):
        t_latent = getattr(teacher_model, "latent_dim", 32)
        # 維度不同時建立投影層，相同時使用 Identity 略過
        if t_latent != student_latent_dim:
            self.projector = nn.Linear(student_latent_dim,
t_latent)
        else:
            self.projector = nn.Identity()

    def distill(self, X_train, epochs=50, alpha=0.5, temper
ature=1.0):
        for batch in loader:
            with torch.no_grad():
                _, z_teacher = teacher(batch)
                z_teacher = z_teacher / temperature      #
Teacher 軟標籤

            x_hat_s, z_student = student(batch)
            z_student_proj = projector(z_student / temperat
ure)  # 投影對齊維度

            L_task    = mse(x_hat_s, batch)              #
重建損失
            L_distill = mse(z_student_proj, z_teacher)   #
蒸餾損失
            loss = alpha * L_task + (1 - alpha) * L_distill
            # alpha 控制任務損失與蒸餾損失的權重比，alpha=0.5 代
表兩者各佔 50%

class ModelCompressor:
    def run_knowledge_distillation(self, student_latent_dim
s=[8, 16, 32]):
        for ld in student_latent_dims:
            distiller = KnowledgeDistiller(self.model, stud
```

```
ent_latent_dim=ld)
            student = distiller.distill(self.X_normal, ...)
            # 評估壓縮率、F1、AUC、推論延遲
```

接著，使用 report_generator.py（指令：python -u "core\report_generator.py"），讀取所有JSON報告，將JSON報告轉成一般人較好閱讀 markdown 格式的結果報告(方便後續放在網頁上)

run_threshold_tuning.py 整合上述所有模組，支援完整的程式執行流程

（訓練→閾值調校→消融實驗→壓縮評估）或是依據需求，可自行選擇執行/測試特定的步驟

二、目前約完成 55 %之進度

比預計的進度 （☑稍微 □嚴重）（☑落後 □超前）。

主要原因：消融實驗與模型壓縮評估的核心邏輯已經實作完成。由於這幾週為期中考週，實驗次數較少、不夠嚴謹。目前初步實驗數據未達預期，後續將針對參數與模型架構進行調整

三、下次主要工作

(進度報告 4）Grad-CAM 視覺化解釋整合與後端開發

- 可解釋性人工智慧（XAI）與熱力圖視覺化整合
- Django 後端系統開發、資料庫管理
- AI Agent（n8n）串接測試

四、遭遇問題(實作、技術或溝通問題等)

請描述遭遇問題/目前解決進度/解決方法/未解決原因：

問題一：非 2^n 輸入尺寸造成 Decoder 輸出維度不符
在消融實驗測試 image_size=24 或 image_size=28 等非 2 的整數冪尺寸時，傳統 ConvTranspose2d（stride=2）在反向上採後因步數累積導致輸出尺寸與輸入不一致，造成損失函數計算錯誤。

解決方法 : Encoder 端改以 AdaptiveMaxPool2d((4,4)) 固定瓶頸尺寸，Decoder 端末層加入 nn.Upsample(size=(image_size, image_size), mode="bilinear") 強制對齊目標尺寸，有效消除因輸入尺寸差異引發的形狀不符問題

問題二：Teacher 模型與 Student 模型的潛在空間維度不匹配

例如 Teacher latent_dim=32、Student latent_dim=16，若直接對兩者的潛在向量計算 MSE 蒸餾損失，會因向量長度不一致而發生維度不匹配錯誤。

解決方法 : KnowledgeDistiller 類別在初始化時透過 getattr 動態讀取 Teacher 的 latent_dim，若與 Student 維度不同，則建立投影層 nn.Linear(student_latent_dim, t_latent)；相同時則使用 nn.Identity() 略過投影。計算蒸餾損失時，先對 Student 的潛在向量施以溫度縮放後送入投影層對齊維度，再與同樣溫度縮放後的 Teacher 潛在向量計算 MSE，成功解決維度不匹配問題

五、老師建議

(包含學長姊的建議)

- 解釋過擬合、過度泛化
- 甘特圖要再大一點
- 要看的是整個系統的架構圖
- 應該要說明提出的內容的選擇與設定會如何影響結果
    - Ex:因為參數值越大會導致實驗結果變大，所以選擇了這個參數
    - Ex:因為使用ＯＯ方法會讓實驗結果被限縮，而ＸＸ方法在這部分能夠解決我們方案的某某需求，所以選擇了這個方法
- 激活函數是什麼? 為何使用 ReLU / GELU / LeakyReLU ?

六、回覆建議

- 過擬合 Overfitting : 模型過度記憶訓練資料的細節與雜訊，導致對未知的新資料缺乏適應力
  (在訓練集表現極佳，但在測試集表現極差)
- 過度泛化 Overgeneralization : 模型的泛化能力太強，把攻擊流量也當成「可以重建的正常流量」，這導致正常與攻擊的重建誤差幾乎相同，閾值無法區分兩者
  (模型太強反而失去偵測異常的能力)

  ※ 邊界損失(Margin Loss):
  邊界損失是一種用來「拉開正常與異常差距」的損失函數，要求攻擊樣本的重建誤差必須大於某個門檻值(堆高重建誤差)
- 激活函數(Activation Function)是應用於神經元輸入的加權總和加上偏差值(bias)的一種數學函數
- 激活函數的核心作用是為神經網路引入非線性轉換(Nonlinear Transformation)。如果沒有非線性激活函數，無論神經網路堆疊了多少層，其整體的數學效果仍然等同於單一的線性轉換，這會導致模型無法解決複雜的真實世界問題。正是因為有了非線性激活函數，神經網路才能夠具備逼近任何連續函數的強大能力

- <u>ReLU (Rectified Linear Unit) :</u>
  - 計算效率高：運算極為簡單，能大幅加快計算速度
  - 避免正值飽和：由於對正值沒有最大輸出限制，能有效減緩梯度下降過程中的問題
  - 缺點：具有神經元死亡 (dying ReLUs) 的風險，當輸入為負值時會永遠輸出 0 且梯度為 0，導致無法繼續學習
- <u>Leaky ReLU :</u>
  - 解決神經元死亡問題：為了解決 ReLU 的缺點而設計。在輸入為負值時保留一個微小的斜率(例如 α)，確保神經元不會完全死亡，讓陷入休眠的神經元仍有機會在未來訓練中甦醒並更新權重

  ※ 神經元死亡(dying ReLUs) : 是使用 ReLU 激活函數時常見的一個缺陷

  發生的原因： 在模型訓練過程中，如果某個神經元的權重被更新(調整)，導致它對於訓練集中的所有資料樣本，其輸入的加權總和皆為負值時，這個神經元就會陷入死亡狀態

  造成的影響： 當輸入為負值時，ReLU 函數會直接輸出 0，且其梯度(導數)也為 0。因為梯度為 0，梯度下降演算法 (Gradient Descent) 就無法再對該神經元發揮作用、更新其權重，導致它永遠只能輸出 0，形同停止了學習

- GELU (Gaussian Error Linear Unit) :
  - **結合正則化與非線性機制 :** 將 Dropout (隨機正則化) 與 ReLU 的特性結合。神經元的輸入會乘上一個由高斯累積分布函數(CDF) 決定的機率值，輸入值越大，保留該值的機率越高
  - **平滑的梯度與曲率 :** 不同於 ReLU 在正值區域為線性且在零點不可導，GELU 具有連續可導的特性，且在所有點上皆具備曲率。這能提供更平滑的梯度變化，使模型在深層架構 (如 Transformer 家族) 中訓練時的收斂過程更加穩定
  - **提升多領域任務效能 :** 在自然語言處理、電腦視覺與語音辨識等任務中，證實表現通常優於標準的 ReLU 與 ELU

※ **正則化 (Regularization) :** 是一種用來約束模型、限制其自由度以防止過度擬合的技術，能提升模型對未知資料的泛化能力

※ **收斂 (Convergence) :** 當模型達到「收斂」時，代表模型內部的參數 (權重與偏差) 已經調整到穩定狀態，找到了針對當前訓練資料的最佳(或次佳) 解答。此時就算繼續增加訓練次數，也不會再為模型帶來明顯的改善

結論 :

選用這三種激活函數進行對比，是因為三者分別代表不同的取捨：ReLU 最有效率、LeakyReLU 修正 Dead Neuron 問題、GELU 提供最平滑的梯度。模型採用 GELU 作為預設激活函數，原因是提升 CNN Autoencoder 的重建品質，以些許計算成本換取更好的特徵萃取能力

> 以上引述自 :
> 1. Aurélien Géron. Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : Concepts, tools, and techniques to build intelligent systems (2nd ed.) .
> 2. Dan. Hendrycks and Kevin. Gimpel, "Gaussian Error Linear Units (GELUs) , 2016.